UTS

PEMBELAJARAN MESIN



NAMA : Salwa Azzahra Rambe

NIM : 202431079

KELAS : E

DOSEN : Rizqia Cahyaningtyas, ST., M.Kom

NO.PC : -

ASISTEN : 1. Muhammad Caisar Gemilang Pratama

2. Benedict Samuel Ardy

3.

4.

INSTITUT TEKNOLOGI PLN

TEKNIK INFORMATIKA

2026/2027

DAFTAR ISI

DAFTAR ISI ii
BAB I 1
PENDAHULUAN 1
1.1 Rumusan Masalah 1
1.2 Tujuan Masalah 1
1.3 Manfaat Masalah 1
BAB II 2
LANDASAN TEORI 2
2.1 Pembelajaran Mesin dan Pembelajaran Terbimbing 2
2.2 Eksplorasi dan Prapemrosesan Data 2
2.3 Naive Bayes 3
2.4 Regresi Linear 4
BAB III 6
HASIL 6
BAB IV 24
PENUTUP 24
DAFTAR PUSTAKA 25

BAB I

PENDAHULUAN

1.1 Rumusan Masalah

1. Bagaimana kondisi dan karakteristik dataset nasabah bank setelah dilakukan EDA dan prapemrosesan data, terutama soal nilai unknown, outlier, dan ketidakseimbangan kelas?
2. Seberapa baik algoritma Naive Bayes memprediksi apakah nasabah berlangganan deposito berjangka, jika dilihat dari accuracy, precision, recall, dan F1-score?
3. Seberapa baik algoritma Regresi Linear memprediksi saldo nasabah (balance), jika dilihat dari MAE, RMSE, dan R^2, dan apakah asumsi regresinya terpenuhi?

1.2 Tujuan Masalah

1. Melakukan EDA dan prapemrosesan pada dataset nasabah bank supaya datanya siap dipakai untuk melatih model.
2. Menerapkan Naive Bayes untuk klasifikasi dan mengetahui seberapa baik hasilnya. Model terbaik yang diperoleh adalah Categorical Naive Bayes dengan accuracy 89,31% dan F1-score kelas yes 0,4759.
3. Menerapkan Regresi Linear untuk memprediksi balance dan memeriksa asumsinya. Hasil yang diperoleh adalah R^2 uji 0,0468, yang naik menjadi 0,0643 setelah nilai balance dibatasi (winsorize).

1.3 Manfaat Masalah

1. Dapat memahami alur pembelajaran mesin dari awal sampai akhir, mulai dari EDA, membersihkan data, membagi data latih dan uji, melatih model, sampai menilai hasilnya.
2. Dapat memahami bahwa accuracy yang tinggi belum tentu berarti model bagus. Pada data yang tidak seimbang ini, tebakan "selalu no" saja sudah mencapai 88,30%, jadi precision, recall, dan F1-score perlu ikut dilihat.
3. Sebelum mengecek asumsi sebelum mempercayai hasil regresi linear, misalnya mengecek multikolinearitas dengan VIF serta normalitas dan homoskedastisitas lewat residual.

BAB II

LANDASAN TEORI

2.1 Pembelajaran Mesin dan Pembelajaran Terbimbing

Pembelajaran mesin atau machine learning adalah cabang kecerdasan buatan yang memungkinkan mesin belajar secara otomatis dari data dan membuat prediksi tanpa harus diprogram dengan aturan yang ditulis satu per satu. Komputer diberi sejumlah contoh, algoritma mencari hubungan yang ada di dalam contoh tersebut, lalu hubungan itu dipakai untuk menebak data baru yang belum pernah dilihat sebelumnya. Karena semua tebakan itu bersumber dari data, kualitas data yang dipakai untuk belajar sangat menentukan kualitas model, sehingga tahap memahami dan merapikan data sama pentingnya dengan memilih algoritma. Metode pembelajaran mesin secara garis besar dibagi menjadi pembelajaran terbimbing dan tidak terbimbing. Pendekatan yang pertama, pembelajaran terbimbing (supervised learning), melatih algoritma dengan data masukan yang sudah diberi label untuk keluaran tertentu. Dari pasangan fitur dan target itulah model belajar memetakan masukan ke keluaran. Masalah yang diselesaikan dengan pendekatan ini terbagi dua. Masalah regresi muncul ketika keluarannya berupa angka kontinu, misalnya harga, sedangkan masalah klasifikasi muncul ketika keluarannya berupa kategori, misalnya ya atau tidak.

2.2 Eksplorasi dan Prapemrosesan Data

Sebelum model dilatih, data perlu dikenali lebih dulu melalui eksplorasi data atau EDA. Pada tahap ini diperiksa bentuk data, tipe tiap kolom, keberadaan nilai kosong atau tidak diketahui, sebaran nilai pada tiap variabel, kemungkinan adanya pencilan, dan hubungan antar variabel. Hasil eksplorasi inilah yang kemudian menentukan langkah prapemrosesan yang diperlukan, bukan sebaliknya. Untuk melakukan prapemrosesan data terdapat empat langkah, yaitu membuang data yang tidak diketahui, mengubah fitur kategorikal menjadi numerik, menyeimbangkan data, dan memilih fitur terbaik. Urutan seperti ini cukup umum dipakai. Fitur kategorikal biasanya berbentuk teks sehingga harus diubah ke bentuk angka sebelum dapat diolah algoritma. Satu hal yang perlu diwaspadai pada pengubahan kategori menjadi variabel dummy adalah jebakan variabel dummy (dummy variable trap), yaitu keadaan ketika jumlah variabel dummy sama dengan jumlah kategori sehingga terjadi informasi yang berulang dan memicu multikolinearitas.

Untuk masalah ketidakseimbangan kelas dapat membandingkan dua teknik. Yang pertama adalah random under-sampling, yaitu mengurangi anggota kelas mayoritas sampai jumlahnya mendekati kelas minoritas. Teknik ini dikategorikan sebagai resampling yang naif karena tidak memakai asumsi atau heuristik tertentu, sehingga mudah dibuat dan cepat dijalankan, tetapi dapat merugikan performa model. Yang kedua adalah SMOTE (Synthetic Minority Oversampling Technique), yang mengambil satu titik dari kelas minoritas lalu membangkitkan titik sintetis baru di sekitarnya menggunakan pendekatan k-nearest neighbor, sehingga sampel baru berada dekat dengan sampel minoritas yang sudah ada. Dari eksperimen mereka, SMOTE dipilih karena

memberi generalisasi yang lebih baik dan mengurangi overfitting. Pemilihan fitur juga dibahas pada penelitian yang sama. Metode filter memberi skor pada tiap fitur terhadap target sebelum proses belajar dimulai, lalu mengambil fitur dengan skor terbaik. Cara ini murah secara komputasi dan membuang fitur yang pengaruhnya kecil.

Metode wrapper bekerja dengan melatih model berulang kali sambil menambah atau mengurangi fitur sampai ditemukan kombinasi terbaik, misalnya lewat forward selection atau backward elimination. Kualitasnya biasanya lebih baik, tetapi biayanya lebih mahal. Pada pengujian mereka dengan k-NN, metode wrapper menghasilkan akurasi tertinggi. Terakhir, data dibagi menjadi data latih dan data uji. Pada penelitian Hayder dkk di tahun 2023, mereka mencoba pembagian 80:20 dan 70:30, dan pada eksperimen mereka porsi data uji 20 persen memberi hasil yang lebih baik dibanding 30 persen. Mereka juga memakai validasi silang lima lipatan (5-fold cross-validation) agar hasil tidak bergantung pada satu kali pembagian data saja.

2.3 Naive Bayes

Naive Bayes adalah metode klasifikasi probabilistik yang dibangun di atas teorema Bayes dan memakai asumsi bahwa antar prediktor tidak saling bergantung. Dengan kata lain, kehadiran satu fitur dianggap tidak dipengaruhi fitur lain selama kelasnya sudah diketahui. Untuk sebuah data dengan fitur X dan kelas C, model menghitung probabilitas posterior tiap kelas berdasarkan teorema Bayes, lalu memilih kelas dengan probabilitas tertinggi.

$$P(C \mid X) = P(X \mid C) \times P(C) / P(X)$$

Pada persamaan di atas, P(C) adalah probabilitas prior kelas yang diperoleh dari proporsi kelas pada data latih, P(X | C) adalah likelihood, yaitu seberapa mungkin fitur X muncul pada kelas C, dan P(X) adalah probabilitas fitur itu sendiri, yang nilainya tidak bergantung pada kelas sehingga dapat diabaikan saat membandingkan kelas. Di sinilah asumsi independensi dipakai. Likelihood merupakan gabungan yang rumit dipecah menjadi perkalian likelihood masing-masing fitur.

$$P(X \mid C) = P(x1 \mid C) \times P(x2 \mid C) \times \ldots \times P(xn \mid C)$$

Pemecahan ini membuat perhitungan jauh lebih ringan, karena model cukup memperkirakan distribusi tiap fitur secara terpisah dan tidak perlu mengestimasi seluruh kombinasi fitur yang jumlahnya sangat besar. Itulah sebabnya Naive Bayes cepat dilatih dan tetap bisa bekerja pada data yang tidak terlalu banyak. Harga yang dibayar adalah asumsinya sendiri. Pada data nyata, fitur hampir selalu berhubungan satu sama lain. Pada data kesehatan, misalnya, tinggi badan dan berat badan jelas saling terkait. Untuk atribut yang seluruhnya kontinu mereka mengusulkan pendekatan transformasi atribut berbasis analisis komponen utama yang memperhitungkan kelas (CWSPCA), dan hasil uji banding mereka menunjukkan bahwa Naive Bayes dapat meningkat banyak bila atributnya ditransformasi dengan tepat.

Ada beberapa varian Naive Bayes, antara lain Gaussian, Bernoulli, dan Multinomial. Perbedaannya terletak pada anggapan tentang bentuk distribusi tiap fitur. Gaussian Naive Bayes menyesuaikan distribusi normal pada tiap kelas, sehingga cocok untuk fitur

numerik. Sedangkan varian Multinomial dan Bernoulli dipakai untuk data diskret, dengan probabilitas fitur diperkirakan dari frekuensi kemunculannya pada tiap kelas. Bila sebuah dataset memuat fitur numerik dan kategorikal sekaligus, pemilihan varian dan cara encoding menjadi keputusan yang perlu dijelaskan pada bab hasil. Ada dua kelemahan utama Naive Bayes adalah asumsi independensi antar fitur dan masalah probabilitas nol, yaitu ketika sebuah fitur muncul pada data uji tetapi tidak pernah ada pada data latih. Karena seluruh likelihood dikalikan, satu nilai nol saja sudah membuat probabilitas total kelas itu nol. Masalah ini ditangani dengan Laplace smoothing, yaitu menghitung probabilitas fitur dari frekuensi relatifnya pada tiap kelas dengan tambahan hitungan kecil sehingga tidak ada probabilitas yang tepat nol. Besarnya penghalusan ini juga dapat dijadikan hiperparameter yang disetel untuk mendapatkan performa terbaik.

Sebagai gambaran kinerjanya, Hayder dkk membandingkan k-NN, decision tree, SVM, dan Naive Bayes pada satu masalah klasifikasi biner. Naive Bayes dengan parameter bawaan hanya mencapai akurasi sekitar 80 sampai 81 persen, sedangkan k-NN dan decision tree berada di kisaran 92 persen dan SVM sekitar 89 persen. Angka ini bukan target yang harus dicapai, tetapi bisa menjadi pembanding yang wajar ketika hasil praktikum dibahas, terutama karena Naive Bayes memang terkenal sederhana dan jarang menjadi yang terbaik pada data dengan banyak fitur yang saling berkaitan.

2.4 Regresi Linear

Regresi Linear dipakai untuk memodelkan hubungan antara satu variabel terikat yang bersifat numerik dan satu atau lebih variabel bebas. Bila variabel bebasnya lebih dari satu, modelnya disebut regresi linear berganda. Bentuk umumnya adalah sebagai berikut.

$$y = \beta 0 + \beta 1 \cdot x1 + \beta 2 \cdot x2 + \ldots + \beta k \cdot xk + \varepsilon$$

Di sini y adalah variabel yang diprediksi, β0 adalah intercept, β1 sampai βk adalah koefisien tiap variabel bebas, dan ε adalah galat atau bagian yang tidak dapat dijelaskan model. Koefisien umumnya diestimasi dengan metode kuadrat terkecil biasa (ordinary least squares atau OLS), yaitu memilih nilai koefisien yang membuat jumlah kuadrat selisih antara nilai aktual dan nilai prediksi sekecil mungkin. Pada hasil model seperti itu, koefisien suatu variabel ditafsirkan sebagai perubahan rata-rata nilai prediksi untuk kenaikan satu satuan variabel tersebut dengan variabel lain dianggap tetap. Intercept tetap menjadi bagian yang dibutuhkan persamaan walaupun nilainya bisa saja tidak bermakna secara praktis, misalnya ketika nilai nol pada seluruh variabel bebas memang tidak mungkin terjadi.

Estimasi dan kesimpulan dari regresi linear baru dapat dipercaya bila beberapa asumsi klasik terpenuhi. Linearitas hubungan antar variabel, independensi pengamatan, homoskedastisitas atau ragam galat yang konstan, normalitas residual, dan tidak adanya multikolinearitas. Pelanggaran terhadap asumsi ini dapat menghasilkan estimasi yang bias dan tidak efisien serta kesimpulan yang keliru. Pemeriksaannya dapat dilakukan dengan uji diagnostik. Multikolinearitas terjadi ketika dua atau lebih variabel bebas berkorelasi tinggi. Akibatnya varians estimasi koefisien membesar, koefisien menjadi tidak stabil, dan pengaruh tiap variabel sulit dipisahkan. Penyebabnya antara lain variabel yang saling tumpang tindih, ukuran sampel yang kecil, kesalahan pengukuran, dan

penggunaan variabel dummy yang berlebihan. Gejala ini dapat dideteksi dengan matriks korelasi, Variance Inflation Factor (VIF), nilai Tolerance, dan Condition Index. Nilai VIF di atas 10, Tolerance di bawah 0,1, atau Condition Index di atas 30 umumnya dianggap menandakan multikolinearitas serius. VIF dihitung dari koefisien determinasi hasil regresi satu variabel bebas terhadap semua variabel bebas lainnya.

$$VIF = 1 / (1 - R^2)$$

Bila multikolinearitas ditemukan, penanganan yang dapat dipilih antara lain membuang salah satu variabel yang berkorelasi tinggi, menggabungkan variabel menjadi satu indeks, mentransformasi variabel, atau memakai regresi yang lebih tahan seperti ridge dan lasso. Ridge menambahkan penalti pada besar koefisien sehingga estimasi lebih stabil, sedangkan lasso memakai penalti yang dapat membuat sebagian koefisien menjadi tepat nol sehingga sekaligus berfungsi sebagai seleksi variabel. Asumsi linearitas diperiksa antara lain dengan memplot residual terhadap nilai prediksi, dan pola yang sistematis seperti lengkungan menandakan bahwa model linear kurang sesuai. Uji formal seperti Ramsey RESET juga dapat dipakai untuk mendeteksi kesalahan spesifikasi model. Jika linearitas dilanggar, pilihan penanganannya mencakup transformasi variabel seperti logaritma atau kuadrat, regresi polinomial, atau metode robust.

Dengan demikian, sebelum model regresi dipercaya, ada beberapa hal yang perlu diperiksa. Variabel yang sebarannya sangat miring atau memiliki pencilan dapat merusak linearitas dan normalitas residual, sehingga transformasi perlu dipertimbangkan. Variabel kategorikal yang diubah menjadi dummy harus ditangani dengan hati-hati agar tidak memicu jebakan variabel dummy.

BAB III

HASIL

Library yang dipakai pada praktikum ini adalah NumPy dan pandas untuk mengolah data, lalu matplotlib dan seaborn untuk membuat grafik. Pada praktikum ini terdapat pengaturan sns.set(style='whitegrid') fungsi nya adalah supaya grafiknya lebih rapi. Di cell ini juga dibuat variabel NIM dan fungsi watermark() yang menempelkan NIM secara transparan di setiap grafik. Di sini dataset dibaca dengan pd.read_csv(). Argumen sep=';' harus ditulis karena kolom pada file ini dipisah dengan titik koma, bukan koma. Setelah itu ukuran data dicetak dan df.head() menampilkan lima baris pertama.

Import Library

```
# Salwa Azzahra Rambe_202431079
import warnings
import os
warnings.filterwarnings('ignore')
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Agar grafik lebih rapi
sns.set(style="whitegrid")
pd.set_option('display.max_columns', None)
pd.set_option('display.width', 200)
```

Baca Dataset

```
# Salwa Azzahra Rambe_202431079
# Baca dataset (pemisah kolom berupa titik koma)
df = pd.read_csv('04_bank_customers.csv', sep=';')

print('Ukuran data (baris, kolom):', df.shape)

# Menampilkan 5 data teratas
df.head()
```

Dataset terbaca dengan 45.211 baris dan 17 kolom. Dari lima baris pertama terlihat bahwa satu baris mewakili satu nasabah, dengan kolom seperti usia, pekerjaan, status pernikahan, pendidikan, saldo, dan hasil kontak kampanye. Kolom terakhir, y, adalah label yang menunjukkan apakah nasabah berlangganan deposito berjangka atau tidak.

```
Ukuran data (baris, kolom): (45211, 17)
```

	age	job	marital	education	default	balance	housing	loan	contact	day	month	duration	campaign	pdays	previous	poutcome	y
0	58	management	married	tertiary	no	2143	yes	no	unknown	5	may	261	1	-1	0	unknown	no
1	44	technician	single	secondary	no	29	yes	no	unknown	5	may	151	1	-1	0	unknown	no
2	33	entrepreneur	married	secondary	no	2	yes	yes	unknown	5	may	76	1	-1	0	unknown	no
3	47	blue-collar	married	unknown	no	1506	yes	no	unknown	5	may	92	1	-1	0	unknown	no
4	33	unknown	single	unknown	no	1	no	no	unknown	5	may	198	1	-1	0	unknown	no

Descriptive Analysis

```
# Salwa Azzahra Rambe_202431079
# Info dataset: jumlah baris, kolom, tipe data, null
df.info()
```

df.info() dipakai untuk melihat tipe data dan jumlah data yang terisi di setiap kolom. Dari sini kita bisa tahu kolom mana yang nanti perlu diubah sebelum dipakai model. Semua 17 kolom punya 45.211 data yang terisi. Ada tujuh kolom bertipe angka (age, balance, day, duration, campaign, pdays, previous) dan sepuluh kolom bertipe teks, termasuk y. Kolom teks ini nanti harus diubah menjadi angka supaya bisa diproses oleh algoritma.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 45211 entries, 0 to 45210
Data columns (total 17 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   age        45211 non-null  int64
 1   job        45211 non-null  object
 2   marital    45211 non-null  object
 3   education  45211 non-null  object
 4   default    45211 non-null  object
 5   balance    45211 non-null  int64
 6   housing    45211 non-null  object
 7   loan       45211 non-null  object
 8   contact    45211 non-null  object
 9   day        45211 non-null  int64
 10  month      45211 non-null  object
 11  duration   45211 non-null  int64
 12  campaign   45211 non-null  int64
 13  pdays      45211 non-null  int64
 14  previous   45211 non-null  int64
 15  poutcome   45211 non-null  object
 16  y          45211 non-null  object
dtypes: int64(7), object(10)
memory usage: 5.9+ MB
```

```
# Salwa Azzahra Rambe_202431079
# Statistik deskriptif untuk kolom numerik
df.describe().T
```

df.describe() menampilkan ringkasan statistik tiap kolom numerik, yaitu jumlah data, rata-rata, simpangan baku, nilai minimum, kuartil, dan nilai maksimum. Hasilnya saya transpose (.T) supaya setiap fitur menjadi satu baris dan lebih mudah dibaca. Pada hasil nya, usia nasabah berkisar dari 18 sampai 95 tahun dengan rata-rata sekitar 40,9 tahun. Rentang balance sangat lebar, dari -8.019 sampai 102.127, dan rata-ratanya (1.362,27) jauh di atas mediannya (448). Ini menandakan sebarannya miring ke kanan. Kolom duration maksimal 4.918 detik dan campaign maksimal 63 kali kontak. Pada pdays, nilai -1 muncul di lebih dari 75% data (kuartil ke-3 masih -1), yang menurut dokumentasi dataset berarti nasabah belum pernah dihubungi di kampanye sebelumnya.

	count	mean	std	min	25%	50%	75%	max
age	45211.0	40.936210	10.618762	18.0	33.0	39.0	48.0	95.0
balance	45211.0	1362.272058	3044.765829	-8019.0	72.0	448.0	1428.0	102127.0
day	45211.0	15.806419	8.322476	1.0	8.0	16.0	21.0	31.0
duration	45211.0	258.163080	257.527812	0.0	103.0	180.0	319.0	4918.0
campaign	45211.0	2.763841	3.098021	1.0	1.0	2.0	3.0	63.0
pdays	45211.0	40.197828	100.128746	-1.0	-1.0	-1.0	-1.0	871.0
previous	45211.0	0.580323	2.303441	0.0	0.0	0.0	0.0	275.0

Adjustment of Variable Types

```
# Salwa Azzahra Rambe_202431079
# Kolom numerik dan kolom kategorikal
fitur_num = ['age', 'balance', 'day', 'duration', 'campaign', 'pdays', 'previous']
fitur_kat = ['job', 'marital', 'education', 'default', 'housing', 'loan', 'contact', 'month', 'poutcome']

# Ubah kolom kategorikal ke tipe category
for col in fitur_kat:
    df[col] = df[col].astype('category')

df.dtypes
```

Pada praktikum ini saya membuat daftar kolom numerik (fitur_num) dan kolom kategorikal (fitur_kat), lalu semua kolom kategorikal diubah ke tipe category dengan astype('category') supaya pandas membacanya sebagai variabel kategori. Kolom y belum diubah karena nanti dijadikan angka 0 dan 1 saat pembagian data. Yang dimana untuk outputnya ada sembilan kolom kategorikal (job, marital, education, default, housing, loan, contact, month, dan poutcome) bertipe category. Tujuh kolom numerik tetap int64, dan y masih bertipe teks.

```
age             int64
job          category
marital      category
education    category
default      category
balance         int64
housing      category
loan         category
contact      category
day             int64
month        category
duration        int64
campaign        int64
pdays           int64
previous        int64
poutcome     category
y              object
dtype: object
```

Detection and Treatment of Missing Data

```
# Salwa Azzahra Rambe_202431079
# Cek nilai kosong dan baris duplikat
print('Jumlah nilai kosong:', df.isnull().sum().sum())
print('Baris duplikat:', df.duplicated().sum())

# Cek nilai 'unknown' pada kolom kategorikal
unk = pd.DataFrame({
    'jumlah_unknown': [(df[c] == 'unknown').sum() for c in fitur_kat],
    'persen': [round((df[c] == 'unknown').mean() * 100, 2) for c in fitur_kat],
    'jumlah_kategori': [df[c].nunique() for c in fitur_kat]
}, index=fitur_kat)
unk
```

Cell ini dipakai untuk mengecek kualitas data. Pertama dihitung jumlah nilai kosong dan baris duplikat. Setelah itu, untuk tiap kolom kategorikal dihitung berapa banyak nilai 'unknown', persentasenya, dan jumlah kategorinya. Nilai 'unknown' dicek terpisah karena bagi pandas itu hanya teks biasa, bukan nilai kosong. Tidak ada nilai kosong dan tidak

ada baris duplikat. Tetapi nilai unknown lumayan banyak di beberapa kolom: job 288 baris (0,64%), education 1.857 baris (4,11%), contact 13.020 baris (28,80%), dan poutcome 36.959 baris (81,75%). Hayder dkk. (2023) membuang data yang tidak diketahui, tetapi di sini baris unknown tidak saya buang karena menghapusnya akan menghilangkan lebih dari 80% data pada poutcome. Jadi unknown saya pertahankan dan diperlakukan sebagai satu kategori sendiri.

```
Jumlah nilai kosong: 0
Baris duplikat: 0
```

	jumlah_unknown	persen	jumlah_kategori
job	288	0.64	12
marital	0	0.00	3
education	1857	4.11	4
default	0	0.00	2
housing	0	0.00	2
loan	0	0.00	2
contact	13020	28.80	3
month	0	0.00	12
poutcome	36959	81.75	4

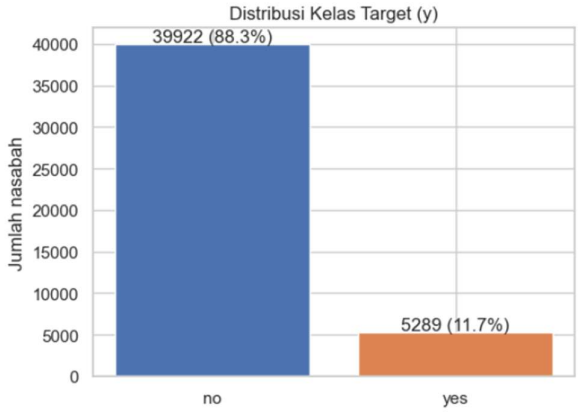
Distribusi Kelas Target

```python
# Salwa Azzahra Rambe_202431079
os.makedirs('fig', exist_ok=True)

# Distribusi target (y)
fig, ax = plt.subplots(figsize=(5.5, 4))
hitung = df['y'].value_counts()
ax.bar(hitung.index, hitung.values, color=['#4C72B0', '#DD8452'])
for i, v in enumerate(hitung.values):
    ax.text(i, v + 300, f'{v} ({v/len(df)*100:.1f}%)', ha='center')
ax.set_title('Distribusi Kelas Target (y)')
ax.set_ylabel('Jumlah nasabah')

plt.tight_layout()
plt.savefig('fig/01_target.png', dpi=150)
plt.show()
print(df['y'].value_counts(normalize=True).round(4))
```

Pada diagram batang dibuat dari value_counts() kolom y untuk melihat seberapa seimbang kedua kelas. Jumlah dan persentase tiap kelas ditulis di atas batang, dan watermark NIM ditempel lewat fungsi watermark(). Kelas no ada 39.922 nasabah (88,3%) dan kelas yes ada 5.289 nasabah (11,7%), jadi datanya jelas tidak seimbang. Akibatnya, model yang selalu menebak 'no' saja sudah mendapat akurasi 88,3% tanpa belajar apa pun. Karena itu accuracy saja tidak cukup untuk menilai model, dan precision, recall, serta F1 untuk kelas yes juga harus dilihat.



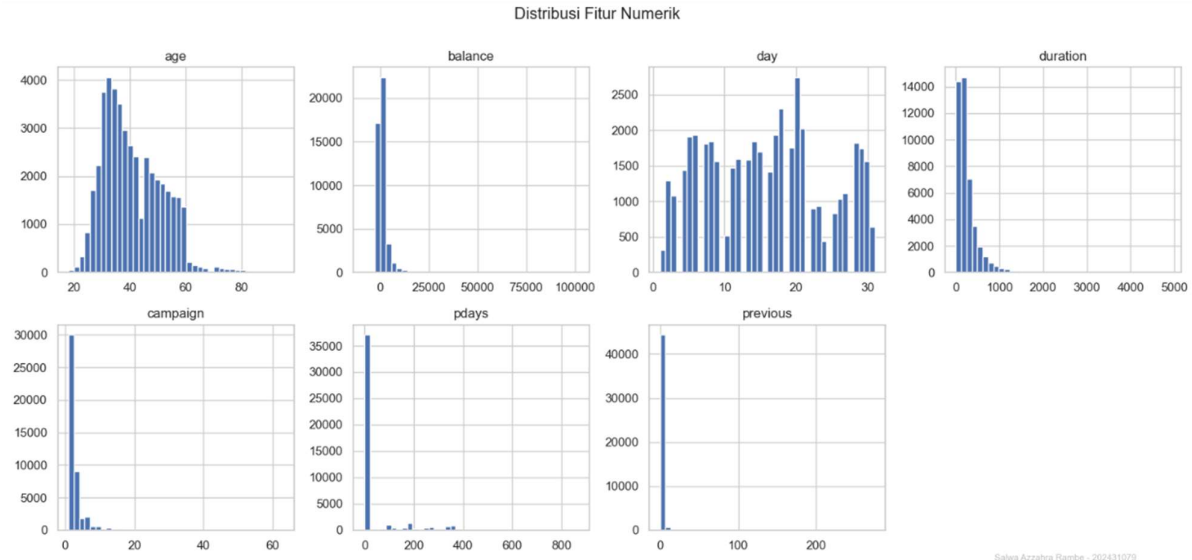

```
y
no     0.883
yes    0.117
Name: proportion, dtype: float64
```

Histogram dibuat untuk ketujuh kolom numerik memakai perulangan pada subplot. Saya juga menghitung skewness dengan .skew() untuk mengetahui seberapa miring sebaran tiap kolom dalam bentuk angka. Hampir semua kolom miring ke kanan. Skewness terbesar ada di previous (41,85) dan balance (8,36), lalu campaign (4,90), duration (3,14), dan pdays (2,62). Age agak miring (0,68), sedangkan day hampir simetris (0,09) karena berupa tanggal dalam bulan. Kemiringan ini penting karena regresi linear mengharapkan residual yang mendekati normal, dan hal itu akan dicek lagi di bagian regresi.

Distribusi Variabel Numerik

```
# Salwa Azzahra Rambe_202431079
# Histogram semua kolom numerik
fig, axes = plt.subplots(2, 4, figsize=(15, 7))
for ax, c in zip(axes.ravel(), fitur_num):
    ax.hist(df[c], bins=40, color='#4C72B0')
    ax.set_title(c)
axes.ravel()[-1].axis('off')
fig.suptitle('Distribusi Fitur Numerik', y=1.0)
watermark(fig)
plt.tight_layout()
plt.savefig('fig/02_hist.png', dpi=130)
plt.show()

# Skewness (kemiringan) tiap kolom
print('Skewness:')
print(df[fitur_num].skew().round(2))
```




```
Skewness:
age          0.68
balance      8.36
day          0.09
duration     3.14
campaign     4.90
pdays        2.62
previous    41.85
dtype: float64
```

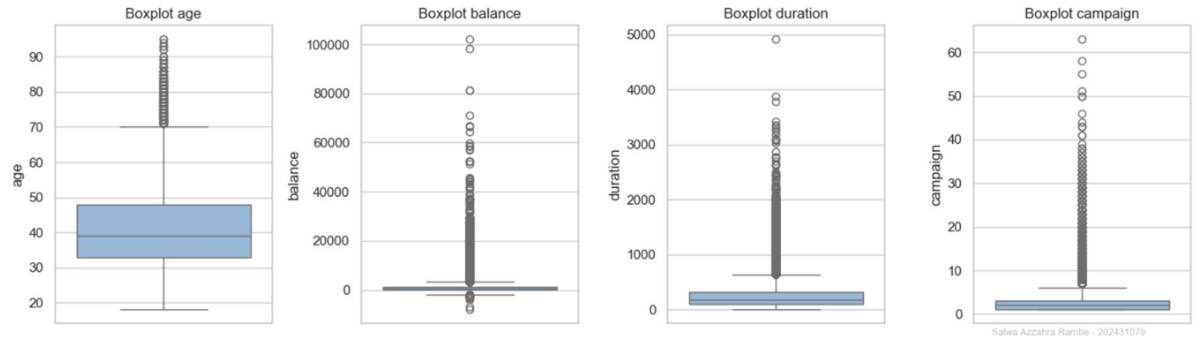
Boxplot dibuat untuk empat kolom yang sebarannya paling lebar. Setelah itu outlier dihitung dengan aturan IQR, yaitu nilai di bawah Q1 - 1,5 x IQR atau di atas Q3 + 1,5 x IQR, lalu hasilnya dirangkum dalam satu tabel. Outlier paling banyak ada di balance (4.729 baris), duration (3.235), campaign (3.064), serta pdays dan previous (masing-masing 8.257). Age hanya 487 dan day tidak punya outlier. Angka pdays dan previous sama karena sebagian besar nasabah memang belum pernah dihubungi, sehingga nasabah yang pernah dihubungi dianggap outlier. Nilai ekstrem ini adalah nasabah sungguhan, bukan salah input, jadi tidak saya hapus. Untuk Naive Bayes pengaruhnya sudah diredam lewat diskretisasi, sedangkan untuk regresi ditangani dengan winsorize di Cell 25.

Identificatioin of Atypical Data

```
# Salwa Azzahra Rambe_202431079
# Boxplot beberapa kolom numerik
fig, axes = plt.subplots(1, 4, figsize=(14, 4))
for ax, c in zip(axes, ['age', 'balance', 'duration', 'campaign']):
    sns.boxplot(y=df[c], ax=ax, color='#8FB8DE')
    ax.set_title('Boxplot ' + c)
watermark(fig)
plt.tight_layout()
plt.savefig('fig/03_box.png', dpi=140)
plt.show()

# Deteksi outlier dengan IQR
jumlah_outlier = {}
for c in fitur_num:
    Q1 = df[c].quantile(0.25)
    Q3 = df[c].quantile(0.75)
    IQR = Q3 - Q1
    jumlah_outlier[c] = ((df[c] < Q1 - 1.5 * IQR) | (df[c] > Q3 + 1.5 * IQR)).sum()

pd.DataFrame({'jumlah_outlier_IQR': jumlah_outlier})
```

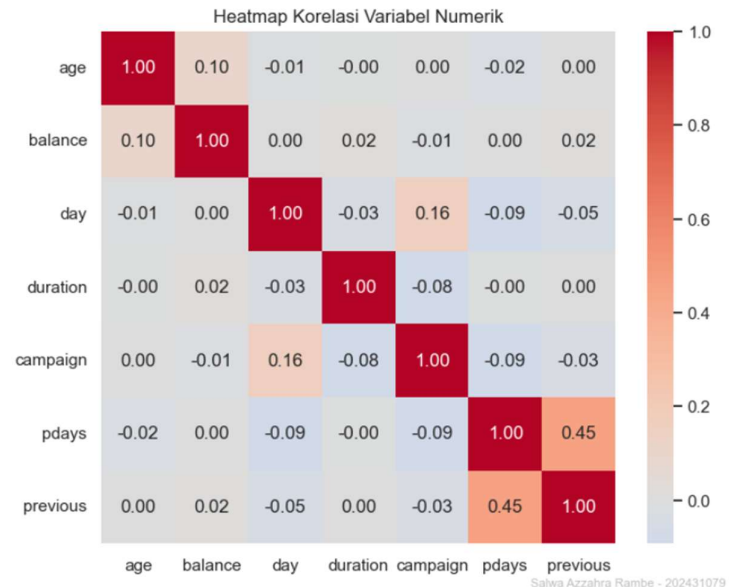


	jumlah_outlier_IQR
age	487
balance	4729
day	0
duration	3235
campaign	3064
pdays	8257
previous	8257

Korelasi antar kolom numerik dihitung dengan df[fitur_num].corr(), lalu digambar sebagai heatmap lengkap dengan angkanya. Tujuannya untuk melihat hubungan linear antar fitur dan mengecek kemungkinan multikolinearitas. Korelasi antar fitur kebanyakan lemah. Pasangan paling kuat adalah pdays dan previous (0,45), lalu day dan campaign (0,16), serta age dan balance (0,10). Tidak ada pasangan yang korelasinya sangat tinggi, jadi multikolinearitas antar fitur numerik sepertinya bukan masalah serius.

Correlation of Variables

```
# Salwa Azzahra Rambe_202431079
# Heatmap korelasi hanya untuk variabel numerik
fig, ax = plt.subplots(figsize=(7, 5.5))
sns.heatmap(df[fitur_num].corr(), annot=True, fmt='.2f', cmap='coolwarm', center=0, ax=ax)
ax.set_title('Heatmap Korelasi Variabel Numerik')
watermark(fig)
plt.tight_layout()
plt.savefig('fig/04_corr.png', dpi=140)
plt.show()
```

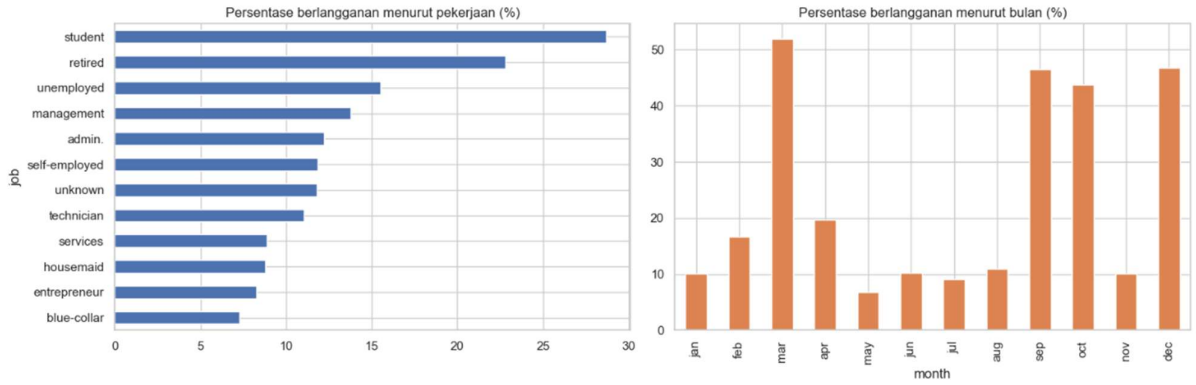


```python
# Salwa Azzahra Rambe_202431079
# Hubungan variabel dengan target: persentase berlangganan menurut pekerjaan dan bulan
df['y_num'] = (df['y'] == 'yes').astype(int)

fig, axes = plt.subplots(1, 2, figsize=(15, 5))
(df.groupby('job')['y_num'].mean().sort_values() * 100).plot.barh(ax=axes[0], color='#4C72B0')
axes[0].set_title('Persentase berlangganan menurut pekerjaan (%)')

urut = ['jan','feb','mar','apr','may','jun','jul','aug','sep','oct','nov','dec']
(df.groupby('month')['y_num'].mean().reindex(urut) * 100).plot.bar(ax=axes[1], color='#DD8452')
axes[1].set_title('Persentase berlangganan menurut bulan (%)')
watermark(fig)
plt.tight_layout()
plt.savefig('fig/05_rate.png', dpi=140)
plt.show()

df = df.drop(columns='y_num')
```



Disini saya membuat kolom bantu y_num (1 untuk yes, 0 untuk no), lalu menghitung rata-ratanya per kelompok dengan groupby sehingga didapat persentase nasabah yang berlangganan. Hasilnya digambar sebagai diagram batang menurut pekerjaan dan menurut bulan (bulan diurutkan dari Januari sampai Desember). Kolom bantu dihapus lagi setelah dipakai. Untuk hasil nya, persentase berlangganan berbeda jauh antar kelompok. Pelajar (28,7%) dan pensiunan (22,8%) paling sering berlangganan, sedangkan blue-collar (7,3%) dan entrepreneur (8,3%) paling rendah. Menurut bulan, Maret (52,0%), Desember (46,7%), September (46,5%), dan Oktober (43,8%) tinggi, tetapi di bulan-bulan itu jumlah nasabah yang dihubungi sedikit (Desember hanya 214 nasabah). Sebaliknya, Mei punya kontak terbanyak (13.766 nasabah) tetapi persentasenya paling rendah (6,7%). Jadi job dan month membawa informasi yang berguna untuk model, tapi persentase di bulan yang datanya sedikit sebaiknya dibaca dengan hati-hati.

Pembagian Data

```python
# Salwa Azzahra Rambe_202431079
from sklearn.model_selection import train_test_split

# nilai 'unknown' dipertahankan sebagai kategori tersendiri
X = df[fitur_num + fitur_kat]
y = (df['y'] == 'yes').astype(int)      # no = 0, yes = 1

# Split data latih 80% dan data uji 20%
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
print('Data latih :', X_train.shape, '| Data uji:', X_test.shape)
print('Proporsi yes (latih):', round(y_train.mean(), 4), '| (uji):', round(y_test.mean(), 4))
```

```
Data latih : (36168, 16) | Data uji: (9043, 16)
Proporsi yes (latih): 0.117 | (uji): 0.117
```

X dibentuk dari daftar fitur_num (tujuh kolom) dan fitur_kat (sembilan kolom) yang dibuat di Cell 5. Nilai unknown dipertahankan sebagai kategori sendiri sesuai keputusan di Cell 6, dan label y diubah menjadi angka (no = 0, yes = 1). Data lalu dibagi 80% untuk

latih dan 20% untuk uji dengan train_test_split. Argumen stratify=y menjaga proporsi kelas tetap sama di kedua bagian, dan random_state=42 membuat hasilnya bisa diulang. Data latih berisi 36.168 baris dan data uji 9.043 baris, masing-masing dengan 16 fitur. Proporsi kelas yes di data latih dan data uji sama, yaitu 0,117, jadi stratifikasi berhasil menjaga ketidakseimbangan kelas tetap seperti data aslinya.

Naive Bayes

```
# Salwa Azzahra Rambe_202431079
from sklearn.preprocessing import StandardScaler
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, confusion_matrix, classification_report)

# Fungsi kecil untuk merangkum metrik klasifikasi (kelas yes)
def evaluasi(nama, y_asli, y_pred):
    return {'Model': nama,
            'Accuracy': accuracy_score(y_asli, y_pred),
            'Precision(yes)': precision_score(y_asli, y_pred),
            'Recall(yes)': recall_score(y_asli, y_pred),
            'F1(yes)': f1_score(y_asli, y_pred)}

# Model A: Gaussian Naive Bayes
# 1) kolom kategorikal diubah menjadi kolom 0/1 (one-hot)
X_a = pd.concat([X[fitur_num], pd.get_dummies(X[fitur_kat]).astype(float)], axis=1)
X_train_a = X_a.loc[X_train.index].copy()
X_test_a = X_a.loc[X_test.index].copy()

# 2) kolom numerik distandardisasi (fit hanya pada data latih)
scaler = StandardScaler()
X_train_a[fitur_num] = scaler.fit_transform(X_train_a[fitur_num])
X_test_a[fitur_num] = scaler.transform(X_test_a[fitur_num])

# 3) latih model dan prediksi data uji
nb_a = GaussianNB()
nb_a.fit(X_train_a, y_train)
pred_a = nb_a.predict(X_test_a)

print('Jumlah fitur setelah encoding:', X_train_a.shape[1])
print(classification_report(y_test, pred_a, target_names=['no', 'yes'], digits=4))
```

Pada Naive Bayes ini pertama dibuat fungsi evaluasi() untuk merangkum accuracy, precision, recall, dan F1 pada kelas yes. Model A dibuat bertahap. Kolom kategorikal diubah menjadi kolom 0/1 dengan pd.get_dummies() (one-hot), lalu kolom numerik distandardisasi dengan StandardScaler yang hanya di-fit pada data latih. Setelah itu GaussianNB dilatih dengan .fit(), yang menganggap tiap fitur berdistribusi normal pada setiap kelas, lalu dipakai menebak data uji dengan .predict(). Hasilnya dirangkum dengan classification_report. Setelah encoding, jumlah fitur menjadi 51. Akurasi Model A adalah 85,48%. Untuk kelas yes, precision 0,4059, recall 0,5198, dan F1 0,4559. Artinya model berhasil menemukan sekitar separuh nasabah yang benar-benar berlangganan, tetapi hanya sekitar 41% tebakan 'yes'-nya yang benar. Untuk kelas no hasilnya jauh lebih bagus dengan F1 0,9162.

```
Jumlah fitur setelah encoding: 51
              precision    recall  f1-score   support

          no     0.9339    0.8992    0.9162      7985
         yes     0.4059    0.5198    0.4559      1058

    accuracy                         0.8548      9043
   macro avg     0.6699    0.7095    0.6860      9043
weighted avg     0.8721    0.8548    0.8624      9043
```

```
# Salwa Azzahra Rambe_202431079
from sklearn.preprocessing import KBinsDiscretizer, OrdinalEncoder
from sklearn.naive_bayes import CategoricalNB

# Model B: Categorical Naive Bayes
# 1) kolom numerik dibagi menjadi 5 interval (bin) berdasarkan kuantil
diskret = KBinsDiscretizer(n_bins=5, encode='ordinal', strategy='quantile')
num_train = diskret.fit_transform(X_train[fitur_num])
num_test = diskret.transform(X_test[fitur_num])

# 2) kolom kategorikal diberi kode angka
kode = OrdinalEncoder()
kat_train = kode.fit_transform(X_train[fitur_kat])
kat_test = kode.transform(X_test[fitur_kat])

# 3) gabungkan kolom numerik dan kategorikal
X_train_b = np.hstack([num_train, kat_train]).astype(int)
X_test_b = np.hstack([num_test, kat_test]).astype(int)
jumlah_kategori = np.maximum(X_train_b.max(axis=0), X_test_b.max(axis=0)) + 1

# 4) latih model (alpha=1 berarti Laplace smoothing) dan prediksi data uji
nb_b = CategoricalNB(alpha=1.0, min_categories=jumlah_kategori)
nb_b.fit(X_train_b, y_train)
pred_b = nb_b.predict(X_test_b)

print(classification_report(y_test, pred_b, target_names=['no', 'yes'], digits=4))
```

Model B memakai cara yang berbeda. Kolom numerik dibagi menjadi lima interval berdasarkan kuantil dengan KBinsDiscretizer, sedangkan kolom kategorikal diberi kode angka dengan OrdinalEncoder, lalu keduanya digabung memakai np.hstack(). Keduanya di-fit hanya pada data latih. Model CategoricalNB dilatih dengan alpha=1,0, artinya Laplace smoothing dipakai. Variabel jumlah_kategori dihitung supaya kategori yang tidak muncul di data latih tidak menyebabkan error. Akurasi Model B mencapai 89,31%. Untuk kelas yes, precision 0,5578, recall 0,4149, dan F1 0,4759. Dibanding Model A, tebakan 'yes' jadi jauh lebih tepat, tetapi recall turun sehingga lebih banyak nasabah yang sebenarnya berlangganan terlewat.

```
              precision    recall  f1-score   support

          no     0.9250    0.9564    0.9405      7985
         yes     0.5578    0.4149    0.4759      1058

    accuracy                         0.8931      9043
   macro avg     0.7414    0.6857    0.7082      9043
weighted avg     0.8821    0.8931    0.8861      9043
```

```
# Salwa Azzahra Rambe_202431079
# Perbandingan kedua model dan baseline (selalu menebak no)
hasil_nb = pd.DataFrame([
    evaluasi('Gaussian NB (Model A)', y_test, pred_a),
    evaluasi('Categorical NB (Model B)', y_test, pred_b),
    {'Model': 'Baseline (selalu tebak no)', 'Accuracy': (y_test == 0).mean(),
     'Precision(yes)': 0.0, 'Recall(yes)': 0.0, 'F1(yes)': 0.0}]).round(4)
hasil_nb
```

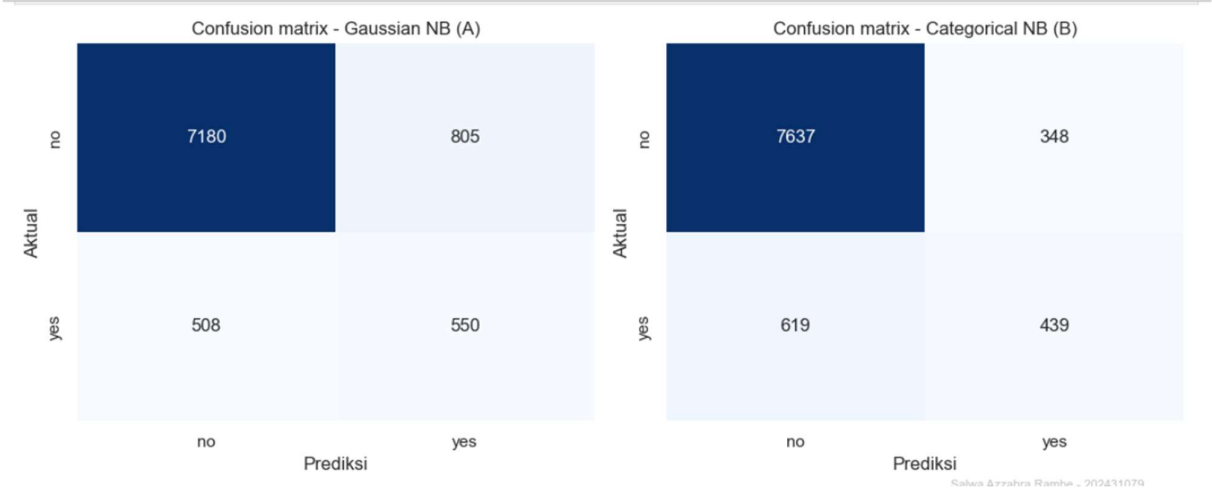
	Model	Accuracy	Precision(yes)	Recall(yes)	F1(yes)
0	Gaussian NB (Model A)	0.8548	0.4059	0.5198	0.4559
1	Categorical NB (Model B)	0.8931	0.5578	0.4149	0.4759
2	Baseline (selalu tebak no)	0.8830	0.0000	0.0000	0.0000

Hasil kedua model dan baseline (selalu menebak no) digabung dalam satu tabel supaya mudah dibandingkan. Model B lebih unggul di accuracy (0,8931 dibanding 0,8548) dan F1 kelas yes (0,4759 dibanding 0,4559), sedangkan Model A lebih tinggi di recall (0,5198

dibanding 0,4149). Perlu dicatat bahwa accuracy Model B hanya sekitar 1 poin persentase di atas baseline 0,8830, jadi akurasi yang tinggi pada data tidak seimbang ini tidak terlalu berarti. F1 kelas yes lebih jujur menggambarkan kemampuan model, dan nilainya masih tergolong sedang. Berdasarkan F1 dan accuracy, Model B saya pilih sebagai model Naive Bayes terbaik.

```
# Salwa Azzahra Rambe_202431079
# Confusion matrix kedua model
fig, axes = plt.subplots(1, 2, figsize=(11, 4.5))
for ax, (nama, pr) in zip(axes, [('Gaussian NB (A)', pred_a), ('Categorical NB (B)', pred_b)]):
    cm = confusion_matrix(y_test, pr)
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False, ax=ax,
                xticklabels=['no', 'yes'], yticklabels=['no', 'yes'])
    ax.set_title('Confusion matrix - ' + nama)
    ax.set_xlabel('Prediksi')
    ax.set_ylabel('Aktual')
watermark(fig)
plt.tight_layout()
plt.savefig('fig/06_cm.png', dpi=140)
plt.show()
```

Confusion matrix kedua model digambar sebagai heatmap dengan confusion_matrix(). Baris menunjukkan kelas aktual dan kolom menunjukkan kelas prediksi. Pada Model A ada 7.180 true negative, 805 false positive, 508 false negative, dan 550 true positive. Pada Model B ada 7.637 true negative, 348 false positive, 619 false negative, dan 439 true positive. Model B jauh lebih jarang salah menandai nasabah sebagai yes (348 dibanding 805), tetapi lebih banyak melewatkan nasabah yang sebenarnya yes (619 dibanding 508). Jadi pilihan model bergantung pada kebutuhan: kalau bank tidak mau kehilangan calon nasabah, Model A lebih cocok, tetapi kalau ingin menghemat biaya menelepon, Model B lebih cocok.



```
# Salwa Azzahra Rambe_202431079
from imblearn.over_sampling import SMOTE

# SMOTE hanya dipakai pada data latih Model A
smote = SMOTE(random_state=42)
X_train_s, y_train_s = smote.fit_resample(X_train_a, y_train)
print('Distribusi latih sebelum SMOTE:', dict(y_train.value_counts()))
print('Distribusi latih sesudah SMOTE:', dict(pd.Series(y_train_s).value_counts()))

nb_s = GaussianNB()
nb_s.fit(X_train_s, y_train_s)
pred_s = nb_s.predict(X_test_a)

hasil_nb2 = pd.DataFrame([
    evaluasi('Gaussian NB (tanpa SMOTE)', y_test, pred_a),
    evaluasi('Gaussian NB (+ SMOTE)', y_test, pred_s),
    evaluasi('Categorical NB (tanpa SMOTE)', y_test, pred_b)]).round(4)
hasil_nb2
```

```
Distribusi latih sebelum SMOTE: {0: np.int64(31937), 1: np.int64(4231)}
Distribusi latih sesudah SMOTE: {0: np.int64(31937), 1: np.int64(31937)}
```

	Model	Accuracy	Precision(yes)	Recall(yes)	F1(yes)
0	Gaussian NB (tanpa SMOTE)	0.8548	0.4059	0.5198	0.4559
1	Gaussian NB (+ SMOTE)	0.7892	0.3117	0.6635	0.4242
2	Categorical NB (tanpa SMOTE)	0.8931	0.5578	0.4149	0.4759

SMOTE dari library imbalanced-learn dipakai untuk menambah sampel sintetis kelas yes pada data latih Model A. SMOTE hanya dipakai di data latih dan data uji tidak disentuh supaya evaluasinya tetap adil. Setelah itu GaussianNB dilatih ulang dan hasilnya dibandingkan dengan model sebelumnya. Sebelum SMOTE, data latih berisi 31.937 kelas no dan 4.231 kelas yes. Sesudahnya kedua kelas masing-masing 31.937. Recall kelas yes naik dari 0,5198 menjadi 0,6635, tetapi precision turun dari 0,4059 menjadi 0,3117, sehingga F1 turun dari 0,4559 menjadi 0,4242 dan accuracy turun menjadi 0,7892. Jadi di percobaan ini SMOTE tidak memperbaiki F1. Hasil ini berbeda dengan temuan Hayder dkk. (2023) yang menyebut SMOTE memberi hasil lebih baik. Dugaan saya, penyebabnya SMOTE dipakai pada fitur hasil one-hot yang isinya 0 dan 1, sehingga titik sintetisnya tidak lagi biner.

```
# Salwa Azzahra Rambe_202431079
# Categorical NB tanpa fitur duration
fitur_num2 = [c for c in fitur_num if c != 'duration']

diskret2 = KBinsDiscretizer(n_bins=5, encode='ordinal', strategy='quantile')
num_train2 = diskret2.fit_transform(X_train[fitur_num2])
num_test2 = diskret2.transform(X_test[fitur_num2])

X_train_c = np.hstack([num_train2, kat_train]).astype(int)
X_test_c = np.hstack([num_test2, kat_test]).astype(int)
jumlah_kategori2 = np.maximum(X_train_c.max(axis=0), X_test_c.max(axis=0)) + 1

nb_c = CategoricalNB(alpha=1.0, min_categories=jumlah_kategori2)
nb_c.fit(X_train_c, y_train)
pred_c = nb_c.predict(X_test_c)

pd.DataFrame([evaluasi('Categorical NB (dengan duration)', y_test, pred_b),
              evaluasi('Categorical NB (tanpa duration)', y_test, pred_c)]).round(4)
```

	Model	Accuracy	Precision(yes)	Recall(yes)	F1(yes)
0	Categorical NB (dengan duration)	0.8931	0.5578	0.4149	0.4759
1	Categorical NB (tanpa duration)	0.8844	0.5097	0.3214	0.3942

```
# Salwa Azzahra Rambe_202431079
# Contoh hasil prediksi Model B pada 10 data uji
contoh = X_test.copy()
contoh['aktual'] = y_test.map({0: 'no', 1: 'yes'})
contoh['prediksi'] = pd.Series(pred_b, index=X_test.index).map({0: 'no', 1: 'yes'})
contoh['peluang_yes'] = nb_b.predict_proba(X_test_b)[:, 1].round(3)
contoh[['age', 'job', 'balance', 'duration', 'aktual', 'prediksi', 'peluang_yes']].head(10)
```

Categorical NB dilatih ulang tanpa kolom duration. Alasannya, durasi panggilan baru diketahui setelah panggilan selesai, sehingga fitur ini tidak tersedia saat bank

memutuskan siapa yang mau dihubungi. Tanpa duration, accuracy turun dari 0,8931 menjadi 0,8844, recall dari 0,4149 menjadi 0,3214, dan F1 dari 0,4759 menjadi 0,3942. Jadi duration sangat membantu model. Tetapi hasil dengan duration terlihat bagus karena memakai informasi yang baru ada setelah hasil panggilan hampir diketahui. Untuk pemakaian nyata, model tanpa duration (F1 0,3942) lebih realistis.

Sepuluh baris pertama data uji ditampilkan beserta label aktual, label prediksi Model B, dan peluang kelas yes dari predict_proba(). Dari sepuluh contoh ini semua prediksi sesuai label aslinya: sembilan nasabah no ditebak no, dan satu nasabah yes ditebak yes. Nasabah yang berlangganan (pensiunan usia 64 tahun dengan durasi panggilan 706 detik) mendapat peluang yes 0,960, sedangkan nasabah no rata-rata mendapat peluang di bawah 0,15. Tapi sepuluh contoh terlalu sedikit untuk mewakili performa model secara keseluruhan.

	age	job	balance	duration	aktual	prediksi	peluang_yes
1392	40	blue-collar	640	347	no	no	0.002
7518	44	technician	378	203	no	no	0.004
12007	31	services	356	228	no	no	0.005
5536	36	blue-collar	655	153	no	no	0.001
29816	34	services	1921	61	no	no	0.012
18275	41	management	1138	283	no	no	0.138
8543	43	entrepreneur	2589	38	no	no	0.002
43965	64	retired	109	706	yes	yes	0.960
28083	35	admin.	1803	61	no	no	0.009
25787	44	services	2279	105	no	no	0.045

Regresi Linear

```
# Salwa Azzahra Rambe_202431079
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# Target regresi: balance (saldo rata-rata tahunan)
fitur_r_num = ['age', 'day', 'duration', 'campaign', 'pdays', 'previous']

# Kolom kategorikal diubah menjadi dummy (drop_first=True agar tidak terjadi dummy trap)
x = pd.get_dummies(df[fitur_r_num + fitur_kat], columns=fitur_kat, drop_first=True).astype(float)
y_reg = df['balance']

# Split data latih dan data uji
x_train, x_test, y_train_r, y_test_r = train_test_split(x, y_reg, test_size=0.2, random_state=42)

print('Jumlah fitur regresi:', x.shape[1])
print('Data latih:', x_train.shape, '| Data uji:', x_test.shape)
print(y_reg.describe().round(2))
```

```
Jumlah fitur regresi: 41
Data latih: (36168, 41) | Data uji: (9043, 41)
count     45211.00
mean       1362.27
std        3044.77
min       -8019.00
25%          72.00
50%         448.00
75%        1428.00
max      102127.00
Name: balance, dtype: float64
```

Untuk Regresi Linear, target yang dipilih adalah balance karena berupa angka kontinu. Kolom numerik dipakai apa adanya, sedangkan kolom kategorikal diubah dengan pd.get_dummies(drop_first=True). Argumen drop_first membuang satu kategori acuan di setiap kolom supaya tidak terjadi dummy variable trap. Data dibagi 80:20 dengan random_state=42 dan tidak distratifikasi karena targetnya berupa angka. Setelah encoding

ada 41 fitur, dengan 36.168 baris latih dan 9.043 baris uji. Ringkasan balance menunjukkan rata-rata 1.362,27, median 448, simpangan baku 3.044,77, minimum -8.019, dan maksimum 102.127. Rata-rata yang jauh di atas median menegaskan bahwa target ini miring ke kanan.

```
# Salwa Azzahra Rambe_202431079
# Buat model regresi linear dan latih
model = LinearRegression()
model.fit(x_train, y_train_r)

# Prediksi data latih dan data uji
pred_tr = model.predict(x_train)
pred_te = model.predict(x_test)

# Evaluasi model
def metrik(y_asli, y_pred):
    mse = mean_squared_error(y_asli, y_pred)
    return {'MAE': mean_absolute_error(y_asli, y_pred), 'MSE': mse,
            'RMSE': np.sqrt(mse), 'R2': r2_score(y_asli, y_pred)}

hasil_lr = pd.DataFrame({'Latih': metrik(y_train_r, pred_tr), 'Uji': metrik(y_test_r, pred_te)}).T.round(4)
print('Intercept:', round(model.intercept_, 2))
hasil_lr
```

```
Intercept: -33.08
```

	MAE	MSE	RMSE	R2
Latih	1475.6535	8.673723e+06	2945.1186	0.0509
Uji	1486.8629	9.338627e+06	3055.9168	0.0468

```
# Salwa Azzahra Rambe_202431079
# Koefisien regresi, diurutkan dari pengaruh terbesar
koef = pd.Series(model.coef_, index=x.columns)
tabel_koef = pd.DataFrame({'koefisien': koef.round(2), 'abs': koef.abs()})
tabel_koef = tabel_koef.sort_values('abs', ascending=False).drop(columns='abs')
tabel_koef.head(12)
```

LinearRegression dilatih pada data latih, lalu dipakai memprediksi data latih dan data uji. Fungsi metrik() menghitung MAE, MSE, RMSE, dan R^2. MAE adalah rata-rata selisih absolut, MSE adalah rata-rata kuadrat selisih, RMSE adalah akar dari MSE (jadi satuannya sama dengan balance), dan R^2 adalah proporsi variasi target yang bisa dijelaskan model. : Di data uji didapat MAE 1.486,86, RMSE 3.055,92, dan R^2 0,0468. Hasil di data latih (R^2 0,0509) hampir sama, jadi model tidak overfitting. Tetapi R^2 sekitar 0,05 berarti model hanya menjelaskan sekitar 4,7% variasi balance, dan RMSE-nya (3.055,92) hampir sama dengan simpangan baku balance (3.044,77). Artinya model ini hampir tidak lebih baik daripada menebak rata-rata untuk semua nasabah. Fitur yang tersedia memang kurang bisa menjelaskan besar kecilnya saldo seseorang.

Koefisien model diambil dari model.coef_, dipasangkan dengan nama fitur, lalu diurutkan berdasarkan nilai mutlaknya untuk melihat fitur mana yang pengaruhnya paling besar. : Koefisien terbesar adalah default_yes (-1.286,92): nasabah yang pernah gagal bayar rata-rata punya saldo sekitar 1.287 lebih rendah dibanding yang tidak, dengan fitur lain dianggap tetap. Pinjaman pribadi (loan_yes, -539,43) juga menurunkan saldo, sedangkan pendidikan tinggi (education_tertiary, +464,10) dan status lajang (marital_single, +399,14) menaikkannya dibanding kategori acuan masing-masing. Arah pengaruhnya masuk akal. Koefisien bulan lebih mencerminkan waktu kampanye daripada sifat nasabah, jadi tidak perlu ditafsirkan terlalu jauh.

	koefisien
default_yes	-1286.92
month_nov	798.19
month_jan	-783.90
month_jul	-589.70
loan_yes	-539.43
education_tertiary	464.10
month_oct	420.43
marital_single	399.14
month_aug	-365.90
month_feb	-329.37
marital_married	324.27
month_may	-301.52

```
# Salwa Azzahra Rambe_202431079
import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor

# Cek multikolinearitas dengan VIF
xc = sm.add_constant(x_train)
vif = pd.Series([variance_inflation_factor(xc.values, i) for i in range(1, xc.shape[1])],
                index=x_train.columns).sort_values(ascending=False).round(2)
print('VIF tertinggi:')
vif.head(8)
```

VIF dihitung untuk setiap fitur dengan statsmodels. Nilai VIF menunjukkan seberapa besar varians koefisien membengkak karena fitur itu berkorelasi dengan fitur lain. VIF tertinggi adalah poutcome_unknown (6,19), lalu month_may (4,77) dan pdays (4,65). Hanya satu fitur yang ada di rentang sedang (5 sampai 10) dan tidak ada yang melewati 10.

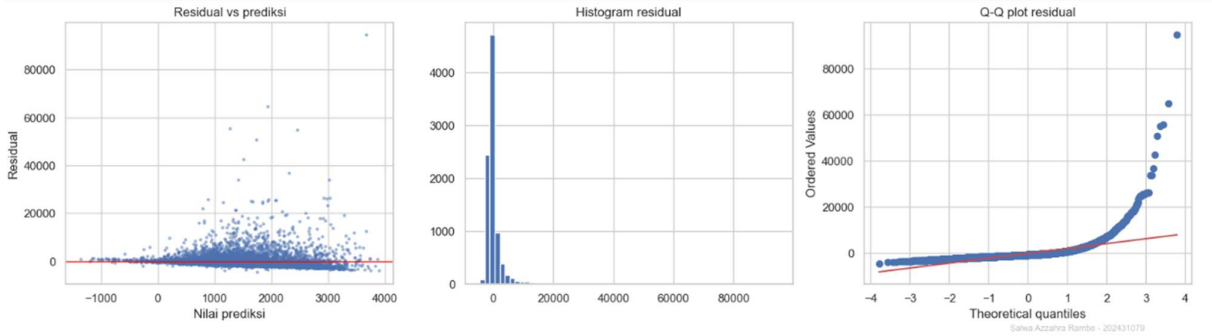
```
VIF tertinggi:
poutcome_unknown      6.19
month_may             4.77
pdays                 4.65
month_jun             3.55
education_tertiary    3.39
month_aug             3.11
month_jul             3.09
job_management        3.03
dtype: float64
```

```
# Salwa Azzahra Rambe_202431079
from scipy import stats

# Cek residual (selisih aktual dan prediksi)
resid = y_test_r - pred_te
fig, axes = plt.subplots(1, 3, figsize=(16, 4.5))
axes[0].scatter(pred_te, resid, s=5, alpha=0.4, color='#4C72B0')
axes[0].axhline(0, color='red', lw=1)
axes[0].set_xlabel('Nilai prediksi')
axes[0].set_ylabel('Residual')
axes[0].set_title('Residual vs prediksi')
axes[1].hist(resid, bins=60, color='#4C72B0')
axes[1].set_title('Histogram residual')
stats.probplot(resid, dist='norm', plot=axes[2])
axes[2].set_title('Q-Q plot residual')
watermark(fig)
plt.tight_layout()
plt.savefig('fig/07_resid.png', dpi=140)
plt.show()
print('Skewness residual:', round(stats.skew(resid), 2))
print('Rata-rata residual:', round(resid.mean(), 2))
```

Residual dihitung sebagai selisih nilai aktual dan prediksi pada data uji, lalu digambar dalam tiga bentuk: scatter residual terhadap nilai prediksi, histogram residual, dan Q-Q plot terhadap distribusi normal. Skewness dan rata-rata residual juga dicetak. Pada hasil, rata-rata residual hanya -7,56, jadi hampir nol. Tapi asumsi yang lain tidak terpenuhi. Di plot residual terhadap prediksi, sebaran residual melebar ketika nilai prediksi membesar

(bentuk corong), yang menandakan ragam galat tidak konstan. Histogramnya sangat miring ke kanan dengan skewness 9,40, dan Q-Q plot menyimpang jauh dari garis lurus di ujung kanan karena ada beberapa residual yang sangat besar. Jadi asumsi normalitas dan homoskedastisitas dilanggar, sesuai dengan teori di Bab II bahwa kemiringan dan outlier pada target bisa merusak asumsi regresi linear.




```
Skewness residual: 9.4
Rata-rata residual: -7.56
```

```python
# Salwa Azzahra Rambe_202431079
# Perbaikan: batasi balance pada persentil 1 dan 99 (winsorize)
b_bawah, b_atas = y_train_r.quantile([0.01, 0.99])
y_train_w = y_train_r.clip(b_bawah, b_atas)
y_test_w = y_test_r.clip(b_bawah, b_atas)

model_w = LinearRegression()
model_w.fit(x_train, y_train_w)
pred_w = model_w.predict(x_test)

banding = pd.DataFrame({'Model awal (balance asli)': metrik(y_test_r, pred_te),
                        'Model winsorize (1-99%)': metrik(y_test_w, pred_w)}).T.round(4)
print('Batas winsorize:', round(b_bawah, 1), 'sampai', round(b_atas, 1))
banding
```

Nilai balance dibatasi di persentil ke-1 dan ke-99 data latih dengan .clip(), lalu model dilatih ulang. Batas yang sama dipakai untuk data uji supaya tidak ada informasi dari data uji yang bocor. Batas winsorize adalah -636,0 sampai 13.107,0. Setelah itu R^2 naik dari 0,0468 menjadi 0,0643, RMSE turun dari 3.055,92 menjadi 2.171,31, dan MAE turun dari 1.486,86 menjadi 1.348,85. Perbaikannya nyata, tetapi perlu diingat dua model ini dievaluasi pada target yang berbeda (balance asli dan balance yang sudah dipotong), jadi perbandingannya tidak sepenuhnya setara. R^2 yang masih di bawah 0,07 menunjukkan model tetap lemah.

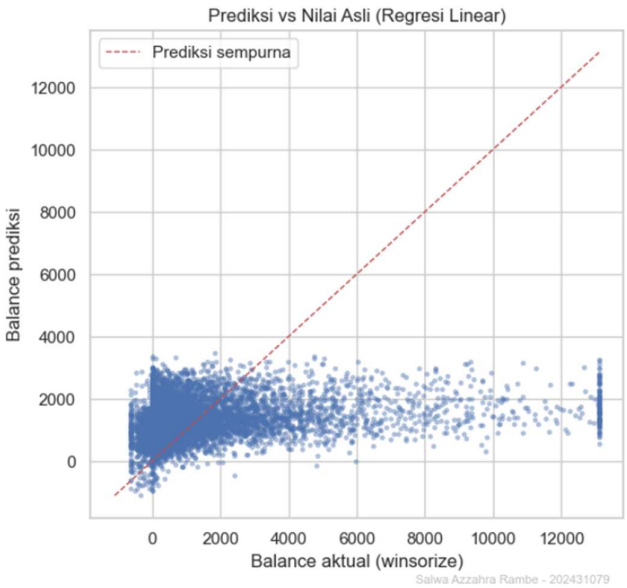
```
Batas winsorize: -636.0 sampai 13107.0
```

	MAE	MSE	RMSE	R2
Model awal (balance asli)	1486.8629	9.338627e+06	3055.9168	0.0468
Model winsorize (1-99%)	1348.8522	4.714588e+06	2171.3102	0.0643

```python
# Salwa Azzahra Rambe_202431079
# Visualisasi prediksi vs nilai asli
fig, ax = plt.subplots(figsize=(6, 5.5))
ax.scatter(y_test_w, pred_w, s=6, alpha=0.35, color='#4C72B0')
batas = [min(y_test_w.min(), pred_w.min()), max(y_test_w.max(), pred_w.max())]
ax.plot(batas, batas, 'r--', lw=1, label='Prediksi sempurna')
ax.set_xlabel('Balance aktual (winsorize)')
ax.set_ylabel('Balance prediksi')
ax.set_title('Prediksi vs Nilai Asli (Regresi Linear)')
ax.legend()
watermark(fig)
plt.tight_layout()
plt.savefig('fig/08_actual_pred.png', dpi=140)
plt.show()

# Contoh 10 hasil prediksi
contoh_r = pd.DataFrame({'balance_aktual': y_test_w.values[:10].round(0),
                         'balance_prediksi': pred_w[:10].round(0)})
contoh_r['selisih'] = (contoh_r['balance_aktual'] - contoh_r['balance_prediksi']).round(0)
contoh_r
```

Scatter plot membandingkan balance aktual (setelah winsorize) dengan hasil prediksi di data uji, dengan garis putus-putus sebagai acuan prediksi sempurna. Sepuluh contoh pertama juga ditampilkan dalam tabel beserta selisihnya. Titik-titik pada plot tersebar jauh dari garis prediksi sempurna. Prediksi model mengumpul di rentang yang sempit, sedangkan nilai aktualnya menyebar jauh lebih lebar. Pada sepuluh contoh, prediksi ada di kisaran 672 sampai 1.874 padahal nilai aktualnya dari 3 sampai 3.644. Selisih terbesar adalah +2.077 (aktual 3.644, prediksi 1.567) dan -1.540 (aktual 127, prediksi 1.667). Model menangkap kecenderungan umum, tetapi belum bisa menangkap variasi saldo antar nasabah.



	balance_aktual	balance_prediksi	selisih
0	580	981.0	-401.0
1	3644	1567.0	2077.0
2	538	1610.0	-1072.0
3	1773	1874.0	-101.0
4	217	1299.0	-1082.0
5	1134	870.0	264.0
6	1085	768.0	317.0
7	127	1667.0	-1540.0
8	3	672.0	-669.0
9	258	1218.0	-960.0

BAB IV

PENUTUP

Naive Bayes bekerja dengan teorema Bayes dan menganggap antar fitur tidak saling bergantung. Karena itu algoritma ini cepat dilatih, tetapi asumsinya sering tidak cocok dengan data nyata. Regresi Linear memodelkan target numerik dari beberapa variabel bebas dengan metode kuadrat terkecil, dan hasilnya baru bisa dipercaya kalau asumsi klasiknya terpenuhi, yaitu linearitas, independensi, homoskedastisitas, normalitas residual, dan tidak ada multikolinearitas. Teori prapemrosesan juga menekankan bahwa nilai unknown, encoding fitur kategorikal, dan ketidakseimbangan kelas harus ditangani lebih dulu sebelum model dilatih.

Dari hasil praktikum pada dataset nasabah bank (45.211 baris dan 17 kolom), hasil EDA menunjukkan tidak ada nilai kosong maupun baris duplikat. Tetapi ada nilai unknown yang cukup banyak, terutama di poutcome (81,75%) dan contact (28,80%), sehingga unknown saya pertahankan sebagai kategori sendiri. Kelas target juga tidak seimbang, yaitu 88,3% no dan 11,7% yes. Selain itu ada banyak outlier, terutama pada balance, tetapi tidak saya hapus karena itu adalah nasabah sungguhan.

Pada Naive Bayes, model terbaik adalah Categorical Naive Bayes dengan accuracy 89,31%, precision 0,5578, recall 0,4149, dan F1-score 0,4759 untuk kelas yes. Hasil ini lebih baik daripada Gaussian Naive Bayes (accuracy 85,48% dan F1-score 0,4559). Tetapi accuracy 89,31% hanya sekitar satu poin di atas tebakan "selalu no" (88,30%), jadi F1-score kelas yes lebih tepat dipakai untuk menilai model. Percobaan SMOTE tidak memperbaiki F1-score (turun menjadi 0,4242), dan model tanpa fitur duration turun menjadi F1-score 0,3942. Artinya duration sangat membantu model, tetapi fitur itu baru diketahui setelah panggilan selesai sehingga kurang realistis untuk prediksi sebelum menelepon.

Pada Regresi Linear dengan target balance, model hanya mendapat R^2 uji 0,0468 dengan RMSE 3.055,92, yang hampir sama dengan simpangan baku balance (3.044,77). Artinya model hanya menjelaskan sekitar 4,7% variasi saldo. Pemeriksaan VIF menunjukkan tidak ada multikolinearitas serius (VIF tertinggi 6,19), tetapi residualnya sangat miring (skewness 9,40) dan sebarannya melebar, sehingga asumsi normalitas dan homoskedastisitas dilanggar. Setelah balance dibatasi pada persentil 1 sampai 99, R^2 naik menjadi 0,0643 dan RMSE turun menjadi 2.171,31, tetapi model tetap lemah.

DAFTAR PUSTAKA

Habiba, U. (2025). Enhancing predictive accuracy in forecasting football World Cup 2022 outcomes. *MBSTU Journal of Science and Technology*, 11(1), 67–74. https://doi.org/10.69728/jst.v11.60

Hayder, I. M., Al Ali, G. A. N., & Younis, H. A. (2023). Predicting reaction based on customer's transaction using machine learning approaches. *International Journal of Electrical and Computer Engineering (IJECE)*, 13(1), 1086–1096. https://doi.org/10.11591/ijece.v13i1.pp1086-1096

Liu, X.-Q., Wang, X.-C., Tao, L., An, F.-X., & Jiang, G.-R. (2024). Alleviating conditional independence assumption of naive Bayes. *Statistical Papers*, 65(5), 2835–2863. https://doi.org/10.1007/s00362-023-01474-5

Najjar, E., & Breesam, A. M. (2023). Supervised machine learning a brief survey of approaches. *Al-Iraqia Journal for Scientific Engineering Research*, 2(4). https://doi.org/10.58564/ijser.2.4.2023.121

Shyrokykh, K., Girnyk, M., & Dellmuth, L. (2023). Short text classification with machine learning in the social sciences: The case of climate change on Twitter. *PLOS ONE*, 18(9), e0290762. https://doi.org/10.1371/journal.pone.0290762